

RBE 550 Final Project:

TAMP for the Assembly of Special Structures

1st Jason Kuehne

RBE

Worcester Polytechnic Institute

Worcester, MA, USA

jkuehne@wpi.edu

2nd Ben Cruse

RBE

Worcester Polytechnic Institute

Worcester, MA, USA

bdcruise@wpi.edu

Abstract—Task and motion planning (TAMP) enables robots to combine high-level symbolic reasoning with continuous geometric planning to complete complex manipulation tasks. This paper presents a modular TAMP pipeline for block-assembly challenges in the Genesis simulator using a Franka Panda arm. The system iteratively (i) lifts the simulator state into PDDL predicates via task-specific state estimation, (ii) generates a PDDL problem and solves it with Fast-Downward, an open source PDDL planner, to obtain a symbolic plan, and (iii) grounds each symbolic action into action primitives built from reusable motion primitives. For geometric execution, we employ OMPL with BIT* to plan joint-space trajectories and apply Jacobian-based end-effector correction to reduce final pose error and improve stacking precision. The pipeline supports multiple tasks, including two ordered three-block towers, adjacency-based special structures, and a two-layer pentagonal “special tower” requiring constrained relative placement and orientation. We report that accurate post-planning correction and careful motion execution are critical to robustness under Genesis physics variability and planner nondeterminism, enabling reliable construction of tall towers (up to 11 blocks) and recovery through replanning after disturbances. We conclude with key implementation challenges and directions for extending the framework to richer symbolic models, improved stability reasoning, and more reliable execution under nondeterministic simulation dynamics.

I. INTRODUCTION

In this project, our main objective was to implement modular task and motion planning capable of directing a robotic arm to complete a variety of structure building challenges, such as constructing a tower of blocks ordered by color and creating a double stacked pentagon of blocks. To solve the problem of grasping and placing blocks in the correct order, we utilize real-time task planning to estimate the current state of the system and then create a plan using fast-downward, a pddl planning library for python. We utilize Genesis to simulate a Franka robotic arm and leverage the premade planners within OMPL to motion-plan to each desired pose. We bridge task and motion planning by creating a series of helper functions that decompose a PDDL plan into a sequence of motion primitives executed within Genesis to complete each task.

The project goals are:

- **Goal 1 (Two Towers):** Build two three-block towers with prescribed color ordering from both provided initial scenes.

- **Goal 2 (5-Block Tower):** Build a stable five-block tower with a specified color order.
- **Goal 3 (Tallest Tower Challenge – Bonus):** Build the tallest stable tower possible, optionally with more than five blocks.
- **Goal 4 (Special Adjacent):** Build a structure resembling a diamond composed of 6 blocks.
- **Goal 5 (Special Tower):** Build a structure consisting of two layers of blocks arranged in a pentagonal shape, with the second layer about 36 degrees out of sync from the first.

II. SYSTEM OVERVIEW

At a high level, the system is composed of:

- 1) **Symbolic abstraction (lifting):** Convert the state of the Genesis scene into a set of logical predicates. The predicates assigned to blocks are determined by their relative location to other blocks and points of interest. These predicates are then converted into a new PDDL problem.
- 2) **Task planning:** Provide a predefined PDDL domain and the generated problem to an external planner (fast-downward) to compute high-level action sequences from a given system state.
- 3) **Motion primitives:** Map each symbolic action (e.g., *PICK*, *STACK*) to a parameterized set of motions to execute the corresponding action in the scene. Each problem has different sets of actions to complete its specialized goal.
- 4) **Motion planning and execution:** Use BITStar from OMPL to plan the necessary trajectories and joint configurations to complete each action.
- 5) **Integration loop:** Estimate the state of the system into a set of predicates, convert the state into a PDDL problem, replan, execute the next action, and repeat.

III. TASK PLANNING WITH PDDL

A. Domain Definition

We begin task planning by turning each building problem into a PDDL domain and a goal. The domain defines:

- **Types:** like `block`, the type representing each cube.

- **Predicates:** such as *on*, *ontable*, *clear*, *holding*, and *handempty*, which describe the full symbolic state of the system at any time.
- **Actions:** such as *PICK*, *PLACE*, *UNSTACK*, and *STACK*, which describe the actions that may be taken to influence the system, and their predicate consequences.

B. Problem Definition

To represent specific building tasks, we define PDDL problems with:

- A set of block objects, e.g.
(:objects r g b c m y - block)
Which are the set of the blocks currently present in a Genesis scene.
- A state specifying the current predicates that apply to each block, e.g.

```
(:init
  (ontable r) (ontable g) (ontable b)
  (ontable c) (ontable m) (ontable y)
  (clear r) (clear g) (clear b)
  (clear c) (clear m) (clear y)
  (handempty)
)
```

- A predefined goal description specifying the desired end state of the system, e.g. a six-block tower:

```
(:goal (and
  (on g r)
  (on b g)
  (on y b)
  (on m y)
  (on c m)
))
```

C. State Estimation

After every action, we assign traits to a data object representing each block present in the scene based on their relative positions to one another. For instance, *block1*, which is directly above *block2* will be assigned the trait *ON* = '*block2*'. Every problem has a different data object class storing traits relevant to the current PDDL domain. For example:

```
@dataclass
class TowerCube:
    """Representation of a block
    in the tower."""
    name: str
    position: np.ndarray
    on: str = None
    clear: bool = True
    ontable: bool = True

@dataclass
class adjacentCube:
    """Representation of a block
    in the adjacency task."""
    name: str
    position: np.ndarray
```

```
NNeighbor: str = None
SNeighbor: str = None
ENeighbor: str = None
WNeighbor: str = None
longDiag: str = None
placed: bool = False
```

Where these data classes are for the tallest tower and adjacent special task respectively. We then parse the traits on each data object into a set of usable PDDL predicates to act as the (:init) section of the problem.

For each task, we have a predefined :goal set to append to the existing predicates to form a complete PDDL problem. For instance:

```
elif self.problem == "adjacency":
    self.adjacent_state_estimation()
    goal_config = [
        ('Nneighbor', 'b2', 'b1'),
        ('Eneighbor', 'b3', 'b1'),
        ('longDiag', 'b4', 'b1'),
        ('Wneighbor', 'b5', 'b4'),
        ('Sneighbor', 'b6', 'b4')
    ]
```

We then lift the resulting block traits and goal into :init and :goal respectively into a single pddl problem to be fed to the planner.

D. Task Planning and Replanning via Fast-Downward

Once the domain and problem are defined and parsed, they are fed into Fast-Downward, an open source Python PDDL planner. Under the hood, we use the *AStar LMCut* algorithm (Landmark cut), which informs the state space search by prioritizing the search of action paths that yield predicates present in the goal state. We then parse the resulting plan into a set of task-space actions to execute. For the tallest tower task, we compute the expected change in predicates that an action will cause and compare it against the actual estimated state, only replanning if there is an unexpected difference between the two. We abandoned this for the other special tasks, as Fast-Downward is efficient enough to complete the replanning process near instantly and we believed our time was better spent elsewhere.

E. Plan Parsing

From the generated solution, we break each action into a list containing the PDDL action and the relevant objects. For instance, the PDDL action (*stack b1 b2*) would be broken into the list [*'stack'*, *'b1'*, *'b2'*]. We then simply find the action primitive related to the PDDL action stored in index 0 and execute it. For instance:

```
elif action_name == "placeeast":
    PLACEEAST(parts[1], parts[2],
              blocks_state, franka, scene)
```

IV. MOTION PLANNING AND ACTION EXECUTION

A. BITStar Motion Planning

We create paths between the current joint configuration and a desired pose using OMPL's BITStar Algorithm. BITStar operates by bounding the sampling region around the start and the goal while prioritizing samples that move the planner towards the goal. The motion planner is set to return a solution as soon as it is found, while never taking more than 6 seconds. Once a solution is found, the resulting path is broken down into a set of waypoints to traverse through. The amount of waypoints can be manually set for each plan, but by default the amount of waypoints is calculated simply by multiplying the angular difference between the initial and final configurations by a predefined constant.

B. Fine Position Adjustment

While BITStar may return a path that leads to a position within acceptable spatial tolerances, the actual final position of the Franka Panda arm typically falls well outside of them, possibly due to minor inaccuracies in the inverse kinematics algorithm or simulated joint error in Genesis. While the shoddy tolerances do not always result in failure for tasks like two towers or adjacency, it makes it impossible to place more than 8 blocks on top of one another without collapse, as the cumulative error between placements quickly leads to the tower becoming unstable; As such, to achieve our desired tower height, we needed functionality to minutely adjust joints of the arm to achieve tolerances within .1 mm. To achieve this, we utilized the Newton-Raphson method to finely tune the position of the EE.

To describe the Newton-Raphson method briefly, it iteratively linearizes the end-effector forward kinematics about the current configuration using the Jacobian and applies incremental joint updates that minimize Cartesian position error at each step. This can be used as a inverse kinematics and path planning method by itself, but in this case we are just using it for fine correction.

After a path is fully executed, we run the Jacobian position correction for several time steps, stopping whenever the EE lands within a desired tolerance of the goal point or it has reached the maximum amount of iterations.

C. Motion Primitives

To simplify motion planning and execution, we define four core motion primitives:

- 1) **move_while_closed:** Given a goal pose, this function calls `plan_path`, verifies that a valid path is returned and that the final pose is close to the target, and executes the path. At each step, the gripper is commanded to maintain a fixed grasping force and position. If the final pose deviates by more than a specified threshold, Jacobian position correction is run for a set number of steps or until the EE is within tolerances.
- 2) **move_while_open:** Similar to `move_while_closed`, but ensures the gripper

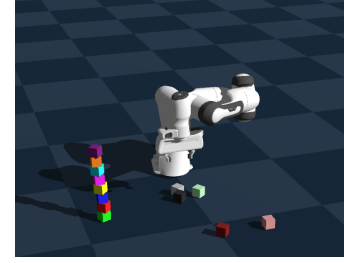


Fig. 1. Rebuilt tower after initial collapse

stays open during motion. To aid in picking blocks under subideal conditions, there is an optional flag to attempt to replan with several different EE orientations about the z axis.

- 3) **close_gripper:** Maintains the arm pose and commands the gripper to close with a target force to grasp a block.
- 4) **open_gripper:** Maintains the arm pose and commands the gripper to open.

D. Action Primitives

To make the action execution pipeline as simple as possible, we create an action primitive for each PDDL action in a domain. These actions, when provided with the aliases of blocks, are capable of planning and moving to positions relative to blocks, enabling the fully dynamic execution of any action with any block. This structure was necessary to create a fully robust replanning pipeline.

V. INTEGRATION AND EXECUTION LOOP

The full TAMP pipeline operates in an iterative plan–execute–replan loop:

- 1) **Lift:** Convert the current Genesis scene state into a symbolic description (PDDL predicates).
- 2) **Task plan:** Generate a PDDL problem file and call Fast-Downward to compute a high-level plan.
- 3) **Ground and execute:** Take the first action in the plan, map it to the appropriate action primitive, and execute it using Genesis motion planning and control.
- 4) **Re-lift and replan:** After execution (or failure), re-lift the scene and re-plan if necessary, until the goal predicates are satisfied or a termination condition is met.

This design allows the system to handle execution failures such as dropped blocks or tower instability by re-synchronizing the symbolic state with the true simulator state.

VI. IMPLEMENTATION OF EACH TASK AND STRUCTURE

A. 11 Block Tower

1) *Design and Implementation:* Our system is capable of reliably stacking 11 blocks. This took hours and hours of fine tuning– differences of less than .000005 meters in desired placement have been observed to make the difference between the 11th block getting stacked or not.

This achievement would not be possible without our Jacobian position correction functionality, as the tolerances of the regular path planner were not sufficient. Additionally, due

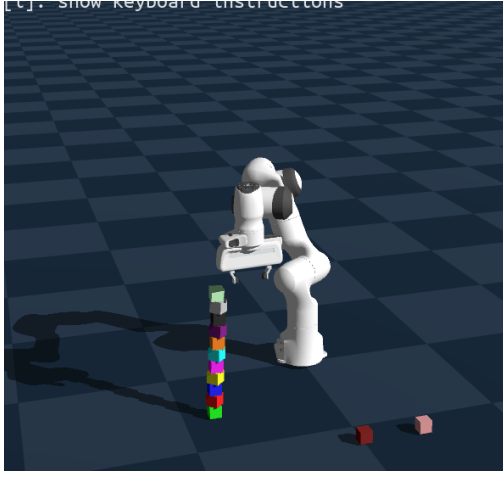


Fig. 2. The 11 block tall tower, moments before it plummets back to Earth

to the instability of rigid body physics in Genesis, we were incentivized to move the arm as fast as possible, and we spent a large amount of time tuning the speed of the arm as it moved from action to action; If it moved too slowly, the tower would fall before all blocks could be placed, and if it moved too quickly, the block would slide in the gripper and result in a run-ending placement. It should be noted that due to the large amount of blocks that fall at once when the tower collapse, the proximity and closeness of the blocks makes it extremely difficult to generate valid trajectories for picking, and as such, the arm can typically only restack a few blocks before a path can no longer be generated without collision.

2) *Chosen PDDL Domain and Problem:* We chose four predicates to represent the state of the tower:

- (on ?x - block ?y - block)- block x is on block y
- (clear ?x - block)- block x has nothing on top, and can be grasped or stacked upon
- (holding ?x - block)- robot is holding block x
- (handempty)- robot hand is empty

These were represented with the data class TowerCube:

```
class TowerCube:
    name: str
    #traits
    position: np.ndarray
    on: str = None
    clear: bool = True
```

The traits were updated according to the follow criteria:

- on: If $\|x_i - x_j + y_i - y_j\| < 3$ cm and $\|z_i - z_j - .04\| < 2$ mm, block i is on block j . This is computed and assigned in `update_stacked`
- clear: If block i is on block j , then block j is not clear. Also updated in `update_stacked`.

We chose three actions to achieve this task:

- **PICK:** The gripper is opened, moved above the target block, lowered into place, then closed to grasp the block. The arm then lifts the block to a safe height (e.g., +0.3 m).
- **STACK:** With a block in hand, the gripper is moved above the target block, lowered by a fixed offset, the gripper is opened to release the block, and then the arm is lifted away.
- **PLACE:** The gripper, with a block in hand, moves to an empty floor location, lowers, and releases the block.

B. Two Towers Task

1) *Design and Implementation:* After completing the tallest tower task, Two Towers was fairly trivial. We were able to reuse the predicates and state checking methods of tallest tower, only augmenting the problem with predicates and state estimation methods necessary to place the initial blocks of each tower in their respective positions.

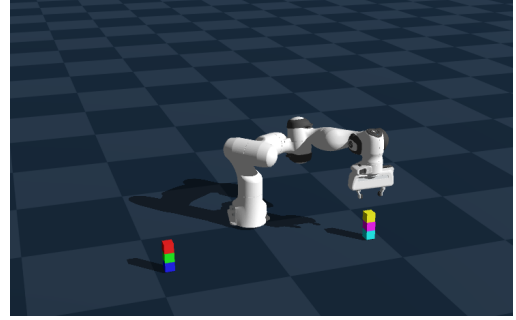


Fig. 3. Two-tower task with towers built in separated target regions.

2) *Chosen PDDL Domain and Problem:* We chose six predicates to represent the state of the tower:

- (on ?x - block ?y - block)- block x is on block y
- (clear ?x - block)- block x has nothing on top, and can be grasped or stacked upon
- (holding ?x - block)- robot is holding block x
- (handempty)- robot hand is empty
- (placedleft ?x)- block x is placed in the left tower base position
- (placedright ?x)- block x is placed in the right tower base position

These were represented with the data class TwoTowerCube:

```
class TwoTowerCube:
    name: str
    position: np.ndarray
    #traits
    on: str = None
    clear: bool = True
    placedleft: bool = False
    placedright: bool = False
```

The traits were updated according to the follow criteria:

- on and clear: Identical to the tallest tower task. Updated in `update_stacked`.

- **placedleft:** If $\|pos_i - (.3, -.45, .02)\| < 4\text{cm}$, block i is placed left. Updated from `estimate_two_towers`
- **placedright:** If $\|pos_i - (.3, .45, .02)\| < 4\text{cm}$, block i is placed right

We chose four actions to achieve this task:

- **PICK:** The gripper is opened, moved above the target block, lowered into place, then closed to grasp the block. The arm then lifts the block to a safe height (e.g., +0.3 m).
- **STACK:** With a block in hand, the gripper is moved above the target block, lowered by a fixed offset, the gripper is opened to release the block, and then the arm is lifted away.
- **PLACELEFT:** The gripper, with a block in hand, moves to the left tower position, lowers, and releases the block.
- **PLACERIGHT:** The gripper, with a block in hand, moves to the right tower position, lowers, and releases the block.

C. Adjacent Task

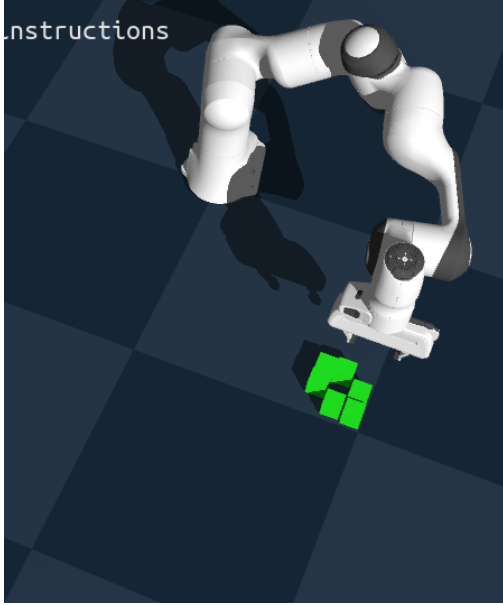


Fig. 4. The constructed Adjacent Task.

1) *Design and Implementation:* While we were able to reuse functions for the picking and initial placement of blocks, we needed to implement new functionality for placing blocks based on a relative lateral offset. To accomplish this, we implemented support for the placement of blocks with varying EE orientations, allowing for blocks to be placed in a row whilst the gripper is oriented perpendicular to the row.

2) *Chosen PDDL Domain and Problem:* We chose six predicates to represent the state of the tower:

- (Nneighbor ?x - block ?y - block)- block x is the north neighbor of block y
- (Eneighbor ?x - block ?y - block)- block x is the East neighbor of block y
- (Sneighbor ?x - block ?y - block)- block x is the South neighbor of block y

- (Wneighbor ?x - block ?y - block)- block x is the West neighbor of block y
- (longDiag ?x - block ?y - block)- block x is along the long NE diagonal of block y
- (handempty)- robot hand is empty
- (holding ?x - block)- robot is holding block x
- (placed ?x - block)- first block has been placed in initial structure position

These were represented with the data class `adjacentCube`:

```
class adjacentCube:
    name: str
    position: np.ndarray
    #traits
    NNeighbor: str = None
    SNeighbor: str = None
    ENeighbor: str = None
    WNeighbor: str = None
    longDiag: str = None
    placed: bool = False
```

The traits were updated according to the follow criteria: (Note: If block 1 is found to be North of block 2, the block 2 is also defined as south of block 1. This bidirectional relationship applies to all relative cardinal placement traits.)

- **NNeighbor:** If $x_i - x_j < .07$ & $x_i - x_j > .02$ & $\|y_i - y_j\| < .03$, then block i is North of block j and block j is South of block i .
- **ENeighbor:** If $y_i - y_j > -.07$ & $y_i - y_j < -.02$ & $\|x_i - x_j\| < .03$, then block i is East of block j and block j is West of block i .
- **SNeighbor:** If $x_i - x_j > -.07$ & $x_i - x_j < -.02$ & $\|y_i - y_j\| < .03$, then block i is South of block j and block j is North of block i .
- **WNeighbor:** If $y_i - y_j < .07$ & $y_i - y_j > .02$ & $\|x_i - x_j\| < .03$, then block i is West of block j and block j is East of block i .
- **longDiag:** If $\|x_i - x_j - .088\| < 2\text{cm}$ & $\|y_i - y_j - .088\| < 2\text{cm}$, block i is the longDiagonal of block j
- **placed:** If $\|pos_i - (.45, .45, .02)\| < 15\text{cm}$, block i is placed in the starting zone.

We chose four actions to achieve this task:

- **PICK:** The gripper is opened, moved above the target block, lowered into place, then closed to grasp the block. The arm then lifts the block to a safe height (e.g., +0.3 m).
- **PLACE:** With a block in hand, the gripper is moved above the struture starting position, lowered by a fixed offset, the gripper is opened to release the block, and then the arm is lifted away.
- **PLACEEAST:** The gripper, with a block in hand, moves to the East (-y) of the target block, lowers, and releases the held block.
- **PLACENORTH:** The gripper, with a block in hand, moves to the North (+x) of the target block, lowers, and releases the held block.

- **PLACEWEST:**The gripper, with a block in hand, moves to the West (+x) of the target block, lowers, and releases the held block.
- **PLACESOUTH:**The gripper, with a block in hand, moves to the South (-x) of the target block, lowers, and releases the held block.
- **PLACELONGDIAG:**The gripper, with a block in hand, .125 m to the NorthWest (+x+y) of a block, lowers, and releases the block.

D. Special Tower Task

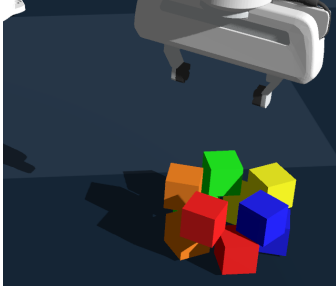


Fig. 5. in-simulation production of the special tower task

1) *Design and Implementation:* Due to the requirement of specific quaternions on each block, along with the tight bounds in which this problem is made, no function could be reused as is. PICK was largely the same, but now adapts the end effector angle to match that of the block being picked up.

2) *Chosen PDDL Domain and Problem:* We chose these predicates to describe the state of the special tower

- (on ?x - block ?y - block)- block x is on block y
- (clear ?x - block)- block x has nothing on top, and can be grasped or stacked upon
- (holding ?x - block)- robot is holding block x
- (handempty)- robot hand is empty
- (placed ?x - block) ; block x is in the right place
- (oriented ?x - block) ; block x is oriented correctly
- (relative ?x - block ?y - block) ; block x is placed relative to block y in special tower formation (clockwise)
- (relative_layered ?x - block ?y - block ?z - block) ; block x is on the second layer relative to y and stacked between y and z

These were represented with the data class SpecialCube. side_stacked was added just for this portion due to relative_layered requiring two blocks to successfully stack, but only being placed relative to one block. The side_stacked block has no effect on the layered block except to be checked if it is in place.

```
class specialCube:
    name: str
    position: np.ndarray
    quat: np.ndarray
    relative: str = None
    relative_layered: str = None
    oriented: bool = False
```

```
placed: bool = False
clear: bool = False
side_stacked: str = None
```

The traits were updated to calculated geometry depending on a reference block and rotation around a reference point. We condense the math above into a function to compute the

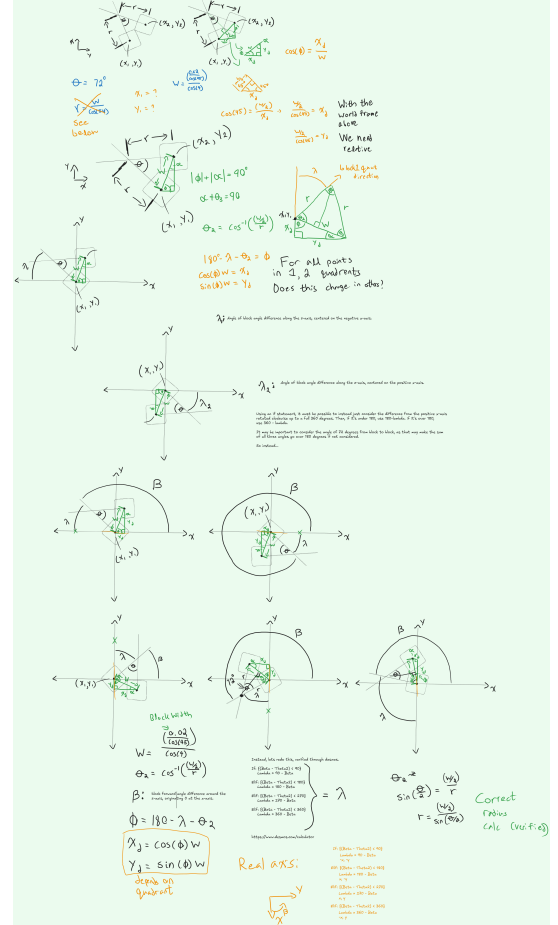


Fig. 6. final page of calculations for the special tower block placement

target position of a new block if placed relative to a chosen reference block.

This function compute_relative has several essential portions. The first part here sets the initial block center point straight-line distance w and the difference in the angle around the z-axis theta_dif. This value is modified depending on whether the new block should be stacked on top or placed on the same level.

```
block_width = 0.04
if stack == False:
    w = ((block_width/math.cos(math.pi/4))
        / (math.cos(9*(math.pi/180))))
    theta_dif = 72*(math.pi/180)
else:
    w = ((block_width/math.cos(math.pi/4))
```



```

    / (math.cos(9*(math.pi/180))))/2
theta_dif = 36*(math.pi/180)

```

Now using these `w` and `theta_dif` values, we can calculate the radius of the circle where all relative blocks are placed on the boundary of. Further angles `theta2` and `beta` are calculated here which will be used later.

```

r = (w/2) / math.sin(theta_dif/2)

theta2 = math.acos((w/2)/r)*(180/math.pi)

beta = R.from_quat(ref_block_quat,
    scalar_first=True).as_euler('xyz')[2]

```

```
beta = beta*(180/math.pi)
```

```

if beta < 0:
    beta = 360 + beta

```

Using the calculated values above, we find a value `lamb` which is used to find `phi`. In this context `phi` is the angle close to the reference block in the triangle of position differences between blocks. These if statements are to ensure `x` and `y` differences are calculated correctly for any possible reference angle.

```

if (beta-theta2) < 90:
    lamb = 180 - beta
    phi = 180 - lamb - theta2
    x_dif = -math.cos(phi*(math.pi/180))*w
    y_dif = -math.sin(phi*(math.pi/180))*w

elif (beta-theta2) < 180:
    lamb = 270 - beta
    phi = 180 - lamb - theta2
    x_dif = math.sin(phi*(math.pi/180))*w
    y_dif = -math.cos(phi*(math.pi/180))*w

elif (beta-theta2) < 270:
    lamb = 360 - beta
    phi = 180 - lamb - theta2
    x_dif = math.cos(phi*(math.pi/180))*w
    y_dif = math.sin(phi*(math.pi/180))*w

elif (beta-theta2) < 360:
    lamb = 450 - beta
    phi = 180 - lamb - theta2
    x_dif = -math.sin(phi*(math.pi/180))*w
    y_dif = math.cos(phi*(math.pi/180))*w

```

If new block is going to be stacked on the second layer, add the height to match. The `target_quat` is then multiplied with reference block quaternion. This converts the desired difference quaternion `target_quat` into the world orientation that the new block will follow. The code described is not shown here due to line length.

Shown below, the quaternion is then verified and ensured that the shorter path is taken.

```

quat_norm = np.linalg.norm(target_quat)
if quat_norm > 0:
    target_quat = target_quat / quat_norm

if target_quat[0] < 0:
    target_quat = -target_quat

return [target_position, target_quat]

```

- on and clear: Identical to the tallest tower task. Updated in `update_stacked`.
- placed: We use $\|block.position - position\| < 0.06$ where `blockposition` is the main block position and `position` is calculated the same fashion as `PLACE_SPECIFIC_ORIENTED` below. For all relative blocks, this is automatically set as true when `relative` or `relative_layered` is also true.
- oriented: This is checked using `z_angle` taken from the block simply as $|z_angle| < \frac{\pi}{16}$ since the `z` angle for the initial block should be 0. For all relative blocks, this is automatically set as true when `relative` or `relative_layered` is also true.
- relative: If the checked block close enough matches the output of `compute_relative` with `stack` as false considering a main block and reference block. The reference block is set as `relative` for the main block.
- relative_layered: If the checked block close enough matches the output of `compute_relative` with `stack` as true considering a main block and reference block. The reference block is then recursively checked for its own relative block, which will then be set as `side_stacked` for the main block.

We chose four actions to achieve this task:

- **PICK:** The gripper is opened, moved above the target block matching the blocks orientation, lowered into place, then closed to grasp the block. The arm then lifts the block to a safe height (e.g., +0.3 m).
- **PLACE_SPECIFIC_ORIENTED:** Considering an initial block `b`

```

b = blocks_state[block]

r =
    ((0.04/math.cos(math.pi/4))
    / (math.cos(9*(math.pi/180))))
    / math.cos(54*(math.pi/180))

theta_block = 0

position =
    np.array(
        [0.6+r*math.cos(theta_block),

```

```

0.3+r*math.sin(theta_block),
0.0]
)

quat =
np.array(R.from_euler('xyz',
    [math.pi,
    0.0,
    theta_block]
).as_quat(scalar_first=True))

x_ee_adjust =
0.003*math.cos(theta_block)

y_ee_adjust =
0.003*math.sin(theta_block)

```

Using these values, the gripper holding the block moves above the target position and orientation that is slightly altered by `ee_adjust` values. It lowers into place then releases slightly while pulling up.

- **PLACE_RELATIVE-ORIENTED:** Considering a main block and reference block. The reference block is used to calculate the desired position and orientation of the main block. The held main block is then moved above the target position at the correct orientation altered by the `ee_adjust` values. It lowers into place then releases slightly while pulling up.
- **STACK_RELATIVE-ORIENTED:** Considering a main block and reference block. The reference block is used to calculate the desired stacked position and orientation of the main block. The held main block is then moved above the target position at the correct orientation altered by the `ee_adjust` values. It lowers into place then releases slightly while pulling up.

VII. CHALLENGES

Throughout development, we encountered several practical challenges:

A. Genesis Physics Engine and Simulation

None of our team members has an NVIDIA GPU, so Genesis runs with significant stuttering and inconsistency. After a brief stutter, blocks in the gripper can jostle, sometimes causing the arm to drop a cube. This results in misaligned placements and reduces tower stability. This was impossible to fix, but we mitigated some of the symptoms of the physics irregularities by assigning a strong force to the gripper, reducing the slippage of blocks. Additionally, we tried to move held blocks to their final destination as quickly as possible, as blocks will always gradually slip out. We attempted to minimize the maximum acceleration experienced by the block by dynamically adjusting joint speed based on the length of a path, helping minimize slippage due to arm movement.

B. Path Planning Nondeterminism

While BITStar can reliably find paths to goals with relatively low final position error, the minor variance in final block placement can result in some tasks failing due to particularly unlucky placement positions. To mitigate this, we implemented the aforementioned Jacobian position correction to ensure that variance in final block placement is minimized. This proved effective and paved the way for the tallest tower and special tower tasks.

C. Random Tree Paths and Jerky Motion

While BITStar is usually very effective in finding a path between two poses, because it returns the first viable solution it generates, paths occasionally are illogical and result in blocks being launched. This was partially mitigated in a number of ways:

- 1) Length Based Waypoint Generation: Speeding or slowing down the arm based on the length of a path tends to stop blocks from being launched even with a real path.
- 2) Intermediate Goals: Planning for poses that are between two desired states allows the planner to solve two simple motion planning problems instead of one complicated one, and typically yields usable results.
- 3) Realtime Planning: Even if a path is very suboptimal and drops a block, if the block lands within the grippers task space, the arm can still recover it and resume the completion of a task.

D. Placing in tight locations

For much of the special tower, space between blocks are tight and prone to collision. This issue can be alleviated by following a few methods. During the `PICK` function, we orient the gripper to match the blocks orientation. Placing the block occurs right above the point where the fingers come in contact with other blocks. When releasing the block during placement, the gripper is opened just enough to release while maintaining Jacobian correction. This allows a final block placement accuracy norm of under 0.008m consistently. Ensuring this placement is accurate is vital for the tower's ability to stand. If the blocks natural orientation isn't possible to be reached by the end effector, it will try at modified orientations until it is successful.

VIII. TEAM CONTRIBUTIONS

We summarize team contributions and estimated person-hours in Table I.

IX. CONCLUSION

We have implemented a modular TAMP pipeline for block stacking tasks in Genesis, combining a PDDL-based task planner with motion primitives and an OMPL-based motion planner. Our system can construct all of the required structures quickly and consistently and replan dynamically should any operation fail. This results in a very efficient and robust pipeline for assembling a variety of structures, which allows

TABLE I
CONTRIBUTION CHART (HOURS BY SUBSECTION)

Section / Subsection	Benjamin Cruse (hrs)	Jason Kuehne (hrs)
1. Motion Execution and PLanning		
1a. Motion Planning (planning.py)	8	0
1b. Motion Primitives (move_while_closed, move_while_open, open_gripper, close_gripper)	12	3
1c. Error Correction (apply_jacobian_correction)	4	0
2. Task Planning and State Estimation		
2a. PDDL Task Planner (fast-downward solver)	6.5	0
2b. State Estimate to PDDL (parse_state_to_pddl, generate_pddl_problem)	2	0
2c. Replanning (replan_from_current_state)	2	0
2d. State Prediction and Comparison (compare_states)	1.5	0
3. Tallest Tower / 5 Block Tower		
3a. Action Primitives	20	0
3b. Realtime Planning	2	0
4. Two Towers		
4a. Action Primitives	.5	1
4b. Realtime Planning	.5	0
5. Adjacent Special Task		
5a. Action Primitives	2.5	0
5b. Realtime Planning	1	0
6. Special Tower Task		
6a. Action Primitives	0	30
6b. Realtime Planning	0	6
7. Final Report		
7a. Introduction	.25	0
7b. System Overview	.25	0
7c. Task Planning with PDDL	1.25	0
7d. Motion Planning & Action Execution	.75	0
7e. Integration and Execution Loop	.15	0
7f. Implementation of Each Task and Structure	1.75	2
7g. Challenges	.5	0.25
7h. Conclusion	.15	0
7i. Appendices	0.25	0.25

for the fairly quick implementation of new block assembly problems and acts as a solid framework for future TAMP problems.

ACKNOWLEDGMENT

We thank the RBE 550 course staff for providing the project framework, initial scenes, and reference code.

REFERENCES

APPENDIX A

FUNCTION REFERENCE

The following functions are enumerated for reference.

A. Motion Planning and Execution Functions

- 1) `move_while_closed(franka, scene, pos, ignore_collisions=True, max_retries=30)`
Uses Genesis motion planning to generate a path from the current configuration to the desired pose. At each pose, the gripper is commanded to close with a fixed force (e.g., 1 N). If the final pose is more than a set threshold from the target (e.g., 10 cm), the planner is called again until a valid plan is produced or the retry limit is reached.
- 2) `move_while_open(franka, scene, pos, max_retries=10)`
Same as `move_while_closed`, but ensures the gripper stays open during motion.
- 3) `close_gripper(franka, scene)`
Holds the arm pose and closes the gripper with force control to grasp a block.
- 4) `open_gripper(franka, scene)`
Holds the arm pose and opens the gripper.
- 5) `open_gripper_slight(franka, scene)`
Opens the gripper slightly while sending no commands to the rest of the arm to preserve Jacobian correction.

B. PDDL Plan Decomposition

- 1) `plan_task(domain_file, problem_file)`
Takes PDDL domain and problem files as input and returns the sequence of actions that solves the problem using Pyperplan's A* solver.
- 2) `execute_pddl_plan(domain_file, problem_file, blocks_state, franka, scene)`
Calls `plan_task`, parses the resulting PDDL action sequence, and executes each action on the arm using the action execution functions and motion primitives.

APPENDIX B

PDDL DOMAIN FILES

A. Tallest Tower

```
(define (domain blocks-world)
  (:requirements :strips :typing)

  (:types block)

  (:predicates
    (on ?x - block ?y - block) ; block x is on block y
    (clear ?x - block) ; block x has nothing on top
    (holding ?x - block) ; robot is holding block x
    (handempty) ; robot hand is empty
  )

  (:action pick
    :parameters (?x - block)
    :precondition (and (clear ?x) (handempty))
    :effect (and (holding ?x) (not (clear ?x)) (not (handempty)))
  )

  (:action stack
    :parameters (?x - block ?y - block)
    :precondition (and (holding ?x) (clear ?y))
    :effect (and (on ?x ?y) (clear ?x) (handempty)
      (not (holding ?x)) (not (clear ?y)))
  )

  (:action place
    :parameters (?x - block)
```

```

    :precondition (holding ?x)
    :effect (and (clear ?x) (handempty) (not (holding ?x)))
  )
)

```

B. *Special Tower*

```

(define (domain special-tower-blocks-world)
  (:requirements :strips :typing)

  (:types block)

  (:predicates
    (on ?x - block ?y - block) ; block x is on block y
    (clear ?x - block) ; block x has nothing on top
    (ontable ?x - block) ; block x is on the table
    (holding ?x - block) ; robot is holding block x
    (handempty) ; robot hand is empty
    (placed ?x - block) ; block x is in the right place
    (oriented ?x - block) ; block x is oriented correctly
    (relative ?x - block ?y - block)
    (relative_layered ?x - block ?y - block ?z - block)
  )

  (:action pick
    :parameters (?x - block)
    :precondition (and (clear ?x) (handempty))
    :effect (and (holding ?x) (not (clear ?x)) (not (ontable ?x))
      (not (handempty)))
  )

  (:action place_specific_oriented
    :parameters (?x - block)
    :precondition (holding ?x)
    :effect (and (clear ?x) (oriented ?x) (placed ?x) (handempty)
      (not (holding ?x)))
  )

  (:action place_relative_oriented
    :parameters (?x - block ?y - block)
    :precondition (and (holding ?x) (placed ?y) (oriented ?y))
    :effect (and (clear ?x) (relative ?x ?y) (placed ?x) (oriented ?x)
      (handempty) (not (holding ?x)))
  )

  (:action stack_relative_oriented
    :parameters (?x - block ?y - block ?z - block)
    :precondition (and (holding ?x) (placed ?y) (placed ?z)
      (oriented ?y) (oriented ?z))
    :effect (and (clear ?x) (placed ?x) (oriented ?x)
      (handempty) (relative_layered ?x ?y ?z) (not (holding ?x)))
  )
)

```

C. *Special Adjacency*

```

(define (domain adjacent-blocks-world-noclear)

```

```

(:requirements :strips :typing)

(:types block)

(:predicates
  (Nneighbor ?x - block ?y - block) ; block x is the north neighbor of block y
  (Eneighbor ?x - block ?y - block) ; block x is the East neighbor of block y
  (Sneighbor ?x - block ?y - block) ; block x is the South neighbor of block y
  (Wneighbor ?x - block ?y - block) ; block x is the West neighbor of block y
  (longDiag ?x - block ?y - block) ; block x is along the long NW diagonal of y
  (handempty) ; robot hand is empty
  (holding ?x - block) ; robot is holding block x
  (placed ?x - block) ; block has been placed in final position
)

(:action pick
  :parameters (?x - block)
  :precondition (and (handempty)) ; hand is empty
  :effect (and (holding ?x) (not (handempty)))
)

(:action place
  :parameters (?x - block)
  :precondition (holding ?x) ; holding x
  :effect (and (handempty) (not (holding ?x)) (placed ?x))
)

(:action placeNorth
  :parameters (?x - block ?y - block)
  :precondition (and (holding ?x) (placed ?y)) ; hand is full
  :effect (and (Nneighbor ?x ?y) (handempty) (not (holding ?x))
    (Sneighbor ?y ?x) (placed ?x))
)

(:action placeEast
  :parameters (?x - block ?y - block)
  :precondition (and (holding ?x) (placed ?y)) ; hand is full
  :effect (and (Eneighbor ?x ?y) (handempty) (not (holding ?x))
    (Wneighbor ?y ?x) (placed ?x))
)

(:action placeSouth
  :parameters (?x - block ?y - block)
  :precondition (and (holding ?x) (placed ?y)) ; hand is full
  :effect (and (Sneighbor ?x ?y) (handempty) (not (holding ?x))
    (Nneighbor ?y ?x) (placed ?x))
)

(:action placeWest
  :parameters (?x - block ?y - block)
  :precondition (and (holding ?x) (placed ?y)) ; hand is full
  :effect (and (Wneighbor ?x ?y) (handempty) (not (holding ?x))
    (Eneighbor ?y ?x) (placed ?x))
)

```

```

(:action placeLongDiag
  :parameters (?x - block ?y - block)
  :precondition (and (holding ?x) (placed ?y)) ; hand is full
  :effect (and (handempty) (not (holding ?x)) (longDiag ?x ?y) (placed ?x))
)
)

```

D. 6 Block Tower

```

(define (domain blocks-world)
  (:requirements :strips :typing)

  (:types block)

  (:predicates
    (on ?x - block ?y - block) ; block x is on block y
    (clear ?x - block) ; block x has nothing on top
    (ontable ?x - block) ; block x is on the table
    (holding ?x - block) ; robot is holding block x
    (handempty) ; robot hand is empty
  )

  (:action pick
    :parameters (?x - block)
    :precondition (and (clear ?x) (ontable ?x) (handempty))
    :effect (and (holding ?x) (not (clear ?x))
      (not (ontable ?x)) (not (handempty)))
  )

  (:action place
    :parameters (?x - block)
    :precondition (holding ?x) ; holding x
    :effect (and (clear ?x) (ontable ?x) (handempty) (not (holding ?x)))
  )

  (:action unstack
    :parameters (?x - block ?y - block)
    :precondition (and (on ?x ?y) (clear ?x) (handempty))
    :effect (and (holding ?x) (clear ?y) (not (on ?x ?y))
      (not (clear ?x)) (not (handempty)))
  )

  (:action stack
    :parameters (?x - block ?y - block)
    :precondition (and (holding ?x) (clear ?y))
    :effect (and (on ?x ?y) (clear ?x) (handempty)
      (not (holding ?x)) (not (clear ?y)))
  )
)

```

E. Two Towers

```

(define (domain towers-world)
  (:requirements :strips :typing)

  (:types block)

  (:predicates

```

```

    (on ?x - block ?y - block) ; block x is on block y
    (clear ?x - block) ; block x has nothing on top
    (holding ?x - block) ; robot is holding block x
    (placedleft ?x - block); block is in right position
    (placedright ?x - block); block is in right position
    (handempty) ; robot hand is empty
)

(:action pick
 :parameters (?x - block)
 :precondition (and (clear ?x) (handempty))
 :effect (and (holding ?x) (not (clear ?x)) (not (handempty)))
)

(:action placeLeft
 :parameters (?x - block)
 :precondition (holding ?x)
 :effect (and (clear ?x) (handempty) (not (holding ?x)) (placedleft ?x))
)

(:action placeRight
 :parameters (?x - block)
 :precondition (holding ?x)
 :effect (and (clear ?x) (handempty) (not (holding ?x)) (placedright ?x))
)

(:action unstack
 :parameters (?x - block ?y - block)
 :precondition (and (on ?x ?y) (clear ?x) (handempty))
 :effect (and (holding ?x) (clear ?y) (not (on ?x ?y))
             (not (clear ?x)) (not (handempty)))
)

(:action stack
 :parameters (?x - block ?y - block)
 :precondition (and (holding ?x) (clear ?y))
 :effect (and (on ?x ?y) (clear ?x) (handempty)
             (not (holding ?x)) (not (clear ?y)))
)
)

```